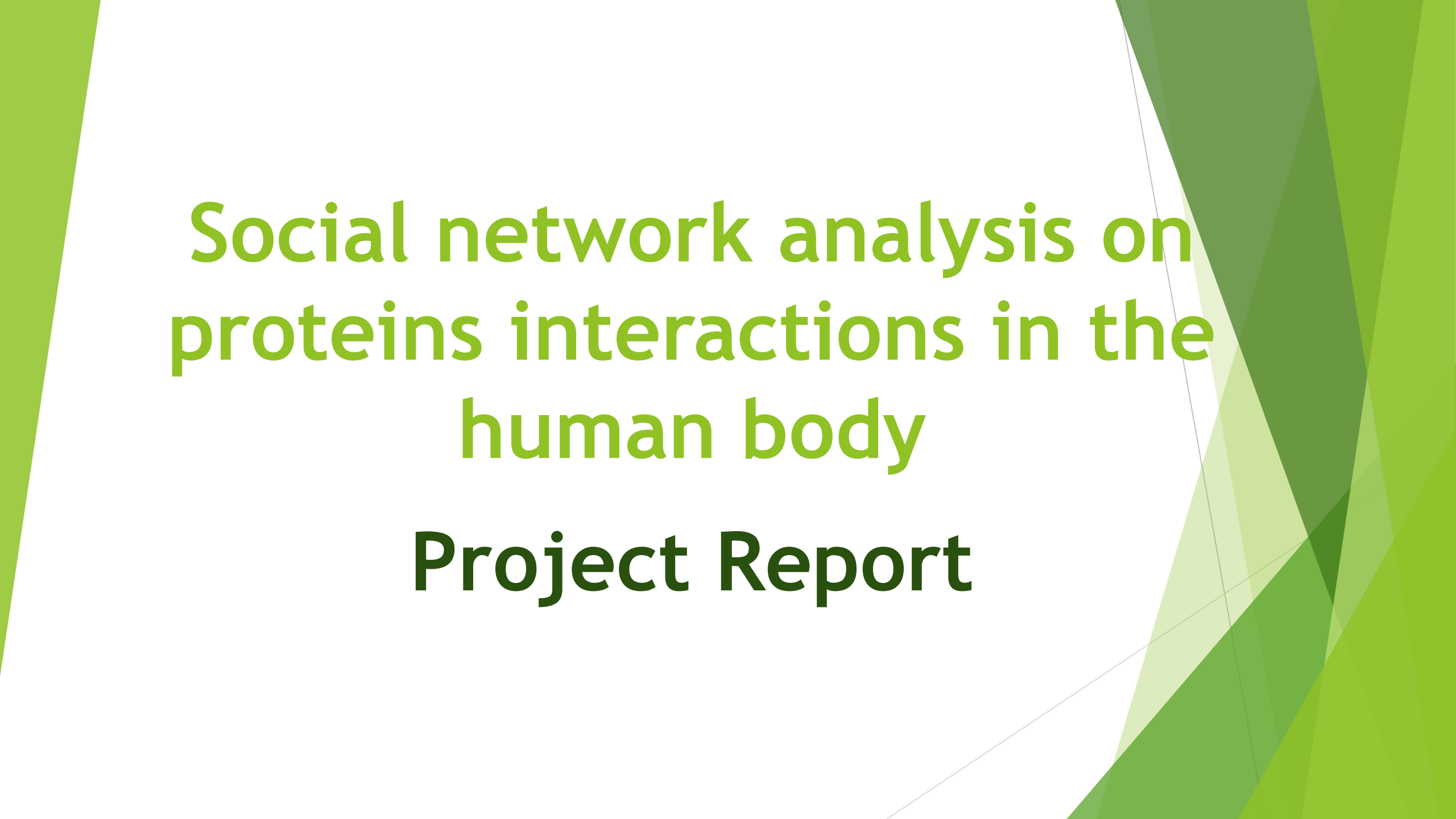


The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Social network analysis on proteins interactions in the human body

Project Report

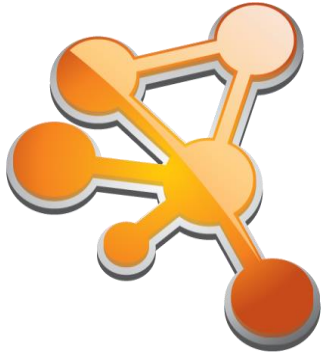
- Aim:

The aim of this analysis is to see which proteins are the most important in the space (in the human body) and their role in fighting viruses and to study the interaction between them and visualize which of the nodes are more important in the proteins interaction network and we will use some of the measures and methods from our social network course

- Hypothesis:

We will use the node centrality analysis to see all four attributes of importance to each node, them being the degree centrality and the betweenness and closeness, and lastly the eigenvector which is the hardest one to get because it needs a bit more calculations and the project will go as follows

- Materials and tools we used for this project are:



Cytoscape: to visualize and to have the option if taking a subset when needed, and also for the ease of use since it has an interface

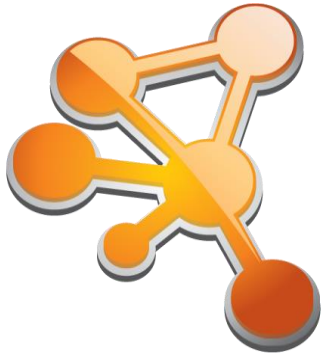


R Studio: we will use R to make sure our calculations are correct and accurate and above all be able to get all the centrality attributes in case Cytoscape fails to do so



As for excel: it's just to view the files in a way everyone can read them and understand them

- What will the programs do:



Cytoscape to open and visualize the XML file for future use, and to be able to make a subset of the network which in our case here is very massive



In R Studio we used some outputs from Cytoscape to make all the calculations needed and how to get the final output



Excel will just show us the results/output and gives a way to interact with them

- Keep in mind!

For clarification: Cytoscape can take care of showing the degree and betweenness and closeness but as mentioned before the eigenvector will need more steps and we will use R for that

And with that we dive onto how we used our tools to analyze this dataset --→:

- What the input of the program will be?

BIOGRID-ORGANISM-Human_Herpesvirus_6B-3.5.174.psi25.xml	39,957	3,447
BIOGRID-ORGANISM-Human_Herpesvirus_6A-3.5.174.psi25.xml	85,639	5,343
BIOGRID-ORGANISM-Human_Herpesvirus_5-3.5.174.psi25.xml	716,175	25,981
BIOGRID-ORGANISM-Human_Herpesvirus_4-3.5.174.psi25.xml	2,367,003	75,909
BIOGRID-ORGANISM-Human_Herpesvirus_3-3.5.174.psi25.xml	38,798	3,114
BIOGRID-ORGANISM-Human_Herpesvirus_2-3.5.174.psi25.xml	44,824	3,441
BIOGRID-ORGANISM-Human_Herpesvirus_1-3.5.174.psi25.xml	1,493,015	45,788
BIOGRID-ORGANISM-Homo_sapiens-3.5.174.psi25.xml	2,049,320,3...	36,855,531
BIOGRID-ORGANISM-Hepatitis_C_Virus-3.5.174.psi25.xml	1,162,091	38,039
BIOGRID-ORGANISM-Glycine_max-3.5.174.psi25.xml	249,185	8,806
BIOGRID-ORGANISM-Gallus_gallus-3.5.174.psi25.xml	2,728,588	73,649
BIOGRID-ORGANISM-Escherichia_coli_K12-3.5.174.psi25.xml	9,341	1,739
BIOGRID-ORGANISM-Escherichia_coli_K12_W3110-3.5.174.psi25.xml	722,149,258	10,594,575
BIOGRID-ORGANISM-Escherichia_coli_K12_MG1655-3.5.174.psi25.xml	10,823,655	219,002

The code that will convert the dataset file into readable Excel files that will see shortly....

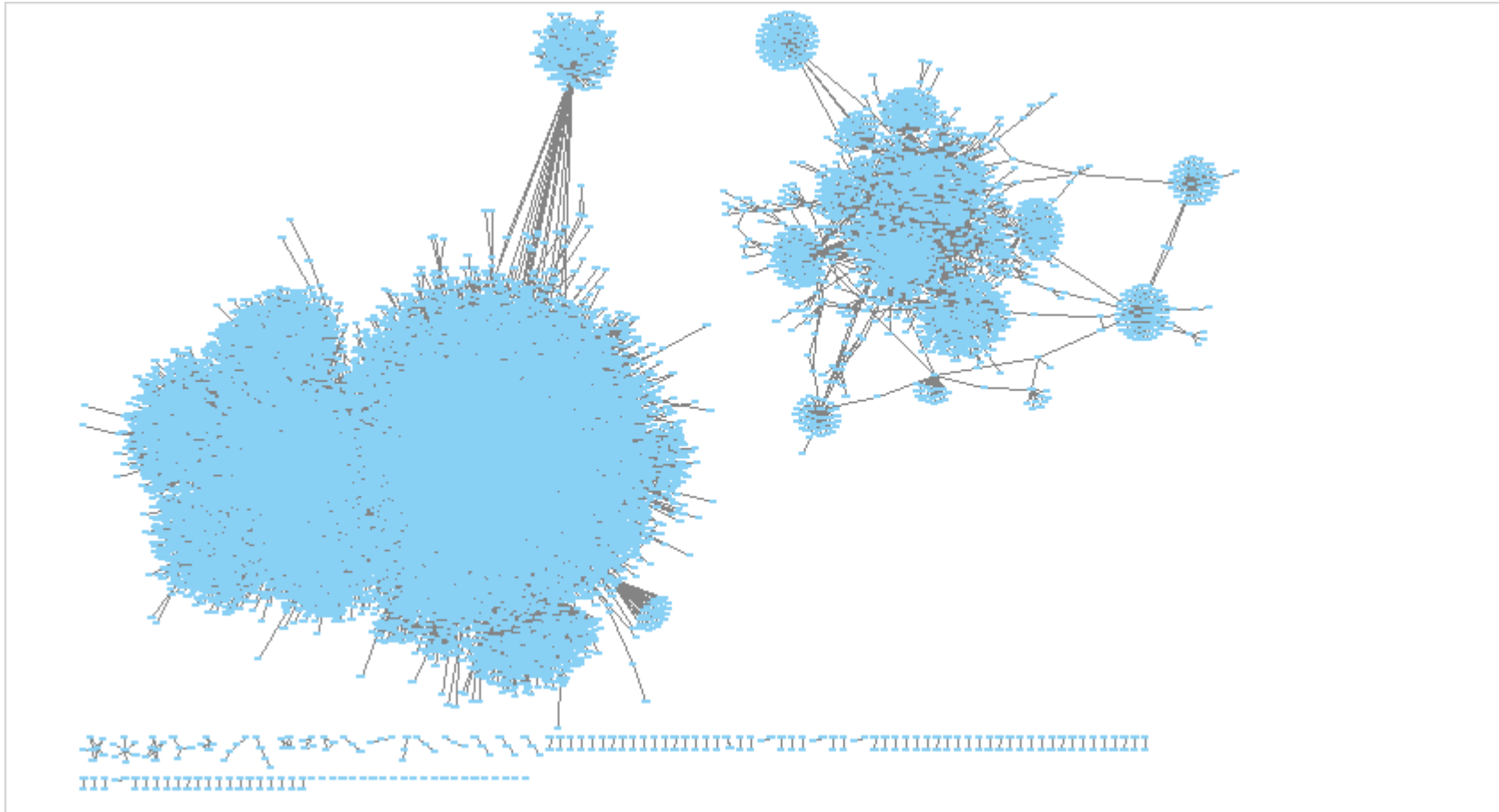
- What the output of the programs will be?

► 1. Data Description

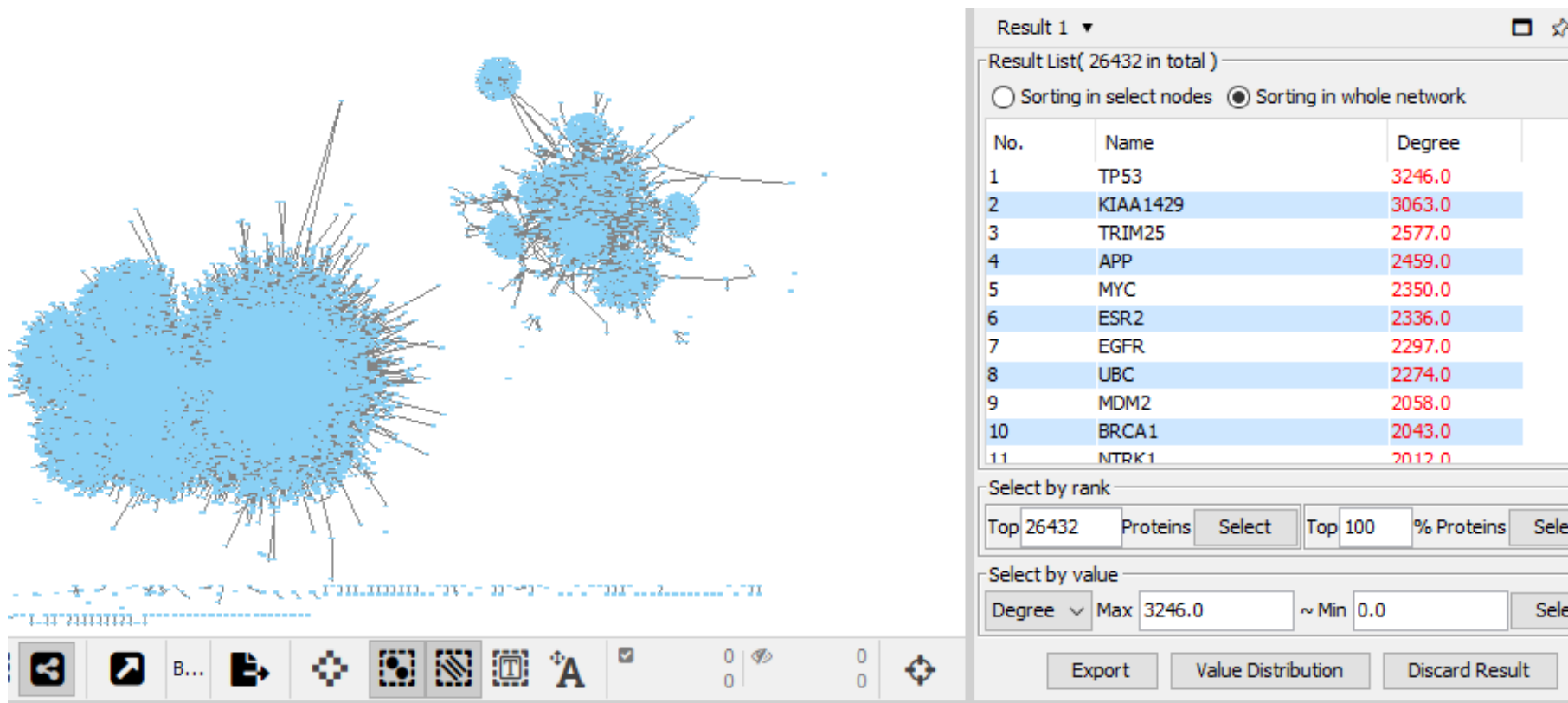
Our data set contains biological interaction data specific to the organism *Homo sapiens* (human) and is formatted in the PSI-MI 2.5 standard for protein-protein interaction data exchange, and was made by BioGRID

Our project steps and methods

We imported the XML file to Cytoscape as a network and it looks as such

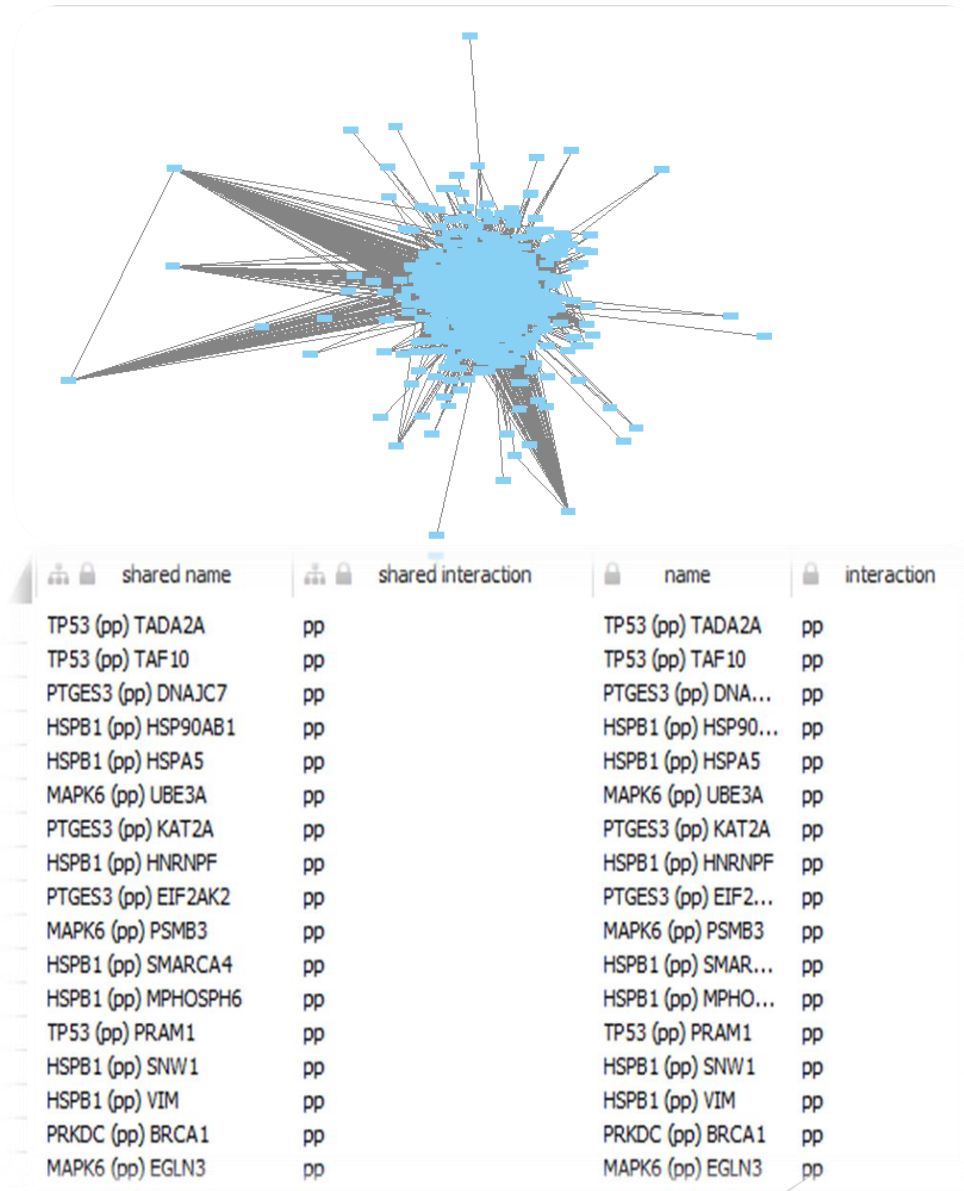


Admittedly the dataset is very massive and our computers can't handle that level of calculations, so we will be taking a subset (even twice) to get a data we can work with we will be using the plugin “CytoNCA” to give us a frame of reference on which node we will use to make the subset



After we are satisfied with our new subNetwork we will extract the “edge table” into a CSV file and save it a place where R can access it (we will store it in documents)

For now other than visualizing our dataset we are pretty much done with Cytoscape and ready to move onto R studio and its codes



- Code Discussion:

Since at the time we didn't know we could just plainly export the CSV file we had to take some "Extreme measures" it being using the following libraries

```
1  #packages installed
2  #first we install the RCy3 package so we can interact with Cytoscape
3  if (!require("BiocManager", quietly = TRUE))
4    install.packages("BiocManager")
5
6  BiocManager::install("RCy3")
7  install.packages("dplyr")
8  install.packages("stringr")
9  install.packages("igraph")
10
11 #packages used
12 require(RCy3)
13 library(dplyr)
14 library(igraph)
15 library(stringr)
16
```

- Code Discussion:

Using Rcy3 package we were able to let R use and control Cytoscape as long as its open on the subnetwork view, we did that and were able to extract/pull all the data we need from the dataset opened in Cytoscape as follows

```
17 #we ping Cytoscape to make sure its connected to R using the Rcy3 package
18 cytoscapePing()
19 cytoscapeVersionInfo()
20 #we load the Cytoscape save file which has the network and all of its sub-networks
21 openSession("proten dataset.cys")#
22 Sys.sleep(1)
23 #we select which network we need to interact with and use for out project
24 setCurrentView("TP53 improved")
25 #e extract the needed data using those commands
26 nodetab=getTableColumns(table="node")
27 edgetab=getTableColumns(table="edge")
28 #after importing the data we want in data frames we will need to clean them so they can be used
29
```

```
Network does NOT exist: 530162d
> cytoscapePing()
You are connected to Cytoscape!
> |
```

- Code Discussion:

Then we will need to do some data cleaning in a way that R can easily get us the results we want using them it will be split into two parts one for the nodes and one for the edge

1. For the nodes

```
29
30 #node wise we only want to take the node names from the nodetab data Frame using
31 newNode <- data.frame(nodetab$`shared name`)
32 colnames(newNode) <- "names"
33 dup_names <- newNode$names[duplicated(newNode$names)] #to keep a count for how many duplicate names
34 ▾ if (length(dup_names) > 0) {
35     newNode <- newNode %>%
36     mutate(names = ifelse(names %in% dup_names, paste0(names, "_", row_number()), names))|
37 ▲ } #to get rid of all the duplicate names
38
```

- Code Discussion:

2. Edges code

```
38
39 #we take a from the edgetab file only the column with edge interaction inside of it
40 new_edge <- data.frame(edgetab$`shared name`)
41 colnames(new_edge) <- PP
42 #we split this one column into two columns one for "from" and the other for "to" edges
43 splitedges <- str_split(new_edge$PP, "pp", simplify = TRUE)
44 #we rename the columns to "from" and "to"
45 colnames(splitedges) <- c("from", "to")
46 #we turn it into a data frame
47 splitedge <- data.frame(splitedges)
48 #and then we clean and make it ready to be used
49 splitedge$from <- sub("\\(", "", splitedge$from)
50 splitedge$to <- sub("\\)", "", splitedge$to)
51 splitedge$from <- sub("\\s*", "", splitedge$from)
52 splitedge$to <- sub("\\s*", "", splitedge$to)
```

- Code Discussion:

After the cleaning is done we then use the (igraph) library to import both data frames to make the g or (graph inside a variable g and g2) as well as getting 3 of the four attributes we wanted

```
54 #with the nodes and edges being ready we start working with igraph
55 #both need to be compatible
56 g <- graph_from_data_frame(d = splitedge, vertices = newNode, directed = FALSE)
57 #we can also only use the edge interaction to make the graph
58 g2 <- graph_from_data_frame(d = splitedge, directed = FALSE)
59
60 #we then use igraph to get the results we need for this dataset
61 node_degree_frame <- data.frame(degree(g))
62 betweenness <- data.frame(betweenness(g))
63 closeness <- data.frame(closeness(g))
64
65 #we then use igraph to get the results we need for this dataset
66 node_degree_frame2 <- data.frame(degree(g2))
67 betweenness2 <- data.frame(betweenness(g2))
68 closeness2 <- data.frame(closeness(g2))
69 |
```


- Code Discussion:

We have one last attribute that we need to get which is the Eigenvector we think using the provided command gives false results so we decided to do it by ourselves, we did that by making the adjacency matrix of both graphs and we imported their degrees into another matrix following the formula provided in the course we should get the final and correct Eigenvector for both done as follows

```
70 #the eigenvector following doesn't give the expected results
71 eigenvector <- eigen_centrality(g)$vector #is wrong
72
73 #so we will need more steps to get it right
74 #first we make the adjacency matrix
75 adj_matrix <- as_adjacency_matrix(g)
76 adj_matrix2 <- as_adjacency_matrix(g2)
77 #put it will be a dgCMatrix which we cant really use so we will need to convert it
78 better_adj_matrix <- as.matrix(adj_matrix)
79 better_adj_matrix2 <- as.matrix(adj_matrix2)
80 #then we need to prepare the node degrees in a matrix as well
81 degree_matrix <- as.matrix(node_degree_frame)
82 degree_matrix2 <- as.matrix(node_degree_frame2)
83
84 #now we multiply both the adjacency matrix and node degree matrix to get the eigenvector
85 eigenvector <- better_adj_matrix %*% degree_matrix
86 eigenvector2 <- better_adj_matrix2 %*% degree_matrix2
```


- Results:

After doing all of this we can now comfortably export our results from R as follows

```
88 #then we export all the needed files
89
90 #for Node degree
91 write.csv(node_degree_frame, file = "Node_Degree.csv", row.names = TRUE)
92 write.csv(node_degree_frame2, file = "Node_Degree2.csv", row.names = TRUE)
93
94 #for betweenness
95 write.csv(betweenness, file = "betweenness.csv", row.names = TRUE)
96 write.csv(betweenness2, file = "betweenness2.csv", row.names = TRUE)
97
98 #for closeness
99 write.csv(closeness, file = "closeness.csv", row.names = TRUE)
100 write.csv(closeness2, file = "closeness2.csv", row.names = TRUE)
101
102 #for eigenvector
103 write.csv(eigenvector, file = "eigenvector.csv", row.names = TRUE)
104 write.csv(eigenvector2, file = "eigenvector2.csv", row.names = TRUE)
```

- Results:

Which will export to us these files (after converting them to excel files)

 betweenness.xlsx	12/14/2023 9:47 PM	Microsoft Excel W...	66 KB
 closeness.xlsx	12/14/2023 9:48 PM	Microsoft Excel W...	55 KB
 eigenvector.xlsx	12/14/2023 9:50 PM	Microsoft Excel W...	51 KB
 Node_Degree.xlsx	12/14/2023 9:45 PM	Microsoft Excel W...	45 KB
 social_output.rar	12/11/2023 8:09 PM	WinRAR archive	199 KB

Before we go to our conclusion Let's talk briefly about what each of the centralities mean given we are so close to us showcasing our final results

- Results(tangent)

1. Degree centrality

Degree centrality is about how many direct connections a node has with other nodes meaning a node becomes more important based on how many connections it has, the greater the better!

- Results(tangent)

2. Betweenness centrality

Betweenness centrality shows us when trying to reach a node from another node, how frequent some nodes will be on that path, the more they show up in the path means some of the other nodes are more likely to use that path as well making that node more important

- Results(tangent)

3. Closeness centrality

Closeness centrality shows us how much are some nodes are like bridges between clusters of nodes, meaning if a node has low closeness that would account into them being closer to most of the network parts making them a reliable path for the entire node

- Results(tangent)

4. Eigenvector centrality

Eigenvector centrality is the importance of a node based on many a node has a connection with other high degree nodes spreading their influence more over that network

- Results:

For the degree centrality (the higher the number the more important a node is)

	A	B
1		degree.g.
2	TP53	576
3	TP53	432
4	MYC	363
5	LARP7	226
6	BRCA1	224
7	HEXIM1	221
8	EGLN3	191
9	COPS5	190
10	CDK2	181
11	CUL7	174
12	ESR1	166
13	SIRT7	166
14	CYLD	161

1940	MDM2_691	0
1941	MAD2L1BP	0
1942	CCL18	0
1943	GPR156	0
1944	ZMIZ1	0
1945	E2	0
1946	FCAMR	0
1947	GPX2	0
1948	USP29	0
1949	TP53INP1	0
1950	USP49	0
1951	APTX	0
1952	MAPK6	0
1953	KIAA0087	0
1954	CCDC106	0
1955	PLK3	0
1956	FAM173A	0
1957	FOXO3_998	0

- Results:

For the betweenness centrality (the higher the number the more important a node is)

	A	B
1		betweenness.g.
2	TP53	488199.4276
3	TP53	218284.7637
4	MYC	118410.8497
5	BRCA1	45827.10585
6	EGLN3	42519.85229
7	HEXIM1	36795.16172
8	XPO1	34366.7141
9	MDM2	32995.9218
10	LARP7	30437.99342
11	CDK2	30361.77521
12	COP55	28590.94424
13	UBC	27384.84858
14	ESR1	24817.25216
15	SIRT7	21582.15358

1943	ACSL4	0
1944	USO1	0
1945	TFRC	0
1946	APOH	0
1947	CAPZB	0
1948	ACKR3	0
1949	THRAP3	0
1950	ANK2	0
1951	IDH3B	0
1952	ANG	0
1953	ANXA3	0
1954	C10orf90	0
1955	POLDIP2	0
1956	HR	0
1957	U14	0

- Results:

For the closeness centrality (the smaller the number the more important a node is) NA means no connection so there will be closeness (∞)

	A	B	C	D
1		closeness.g.		
2	TP53	0.000271887	SULT1E1	NA
3	MYC	0.000241313	FOXJ3	NA
4	TP53	0.000237079	U14_127	NA
5	BRCA1	0.000223814	TRIM59	NA
6	HEXIM1	0.000223514	DLEU1	NA
7	LARP7	0.000222816	U14_132	NA
8	COPS5	0.000218723	CELA2B	NA
9	CUL7	0.000216826	RFFL	NA
10	CDK2	0.000216638	FOXO3_262	NA
11	SIRT7	0.000215054	TRIM45	NA
12	ESR1	0.000214777	NDRG4	NA
13	EGLN3	0.000214684	NLK	NA
14	RNF2	0.000213402	PCDHA4	NA

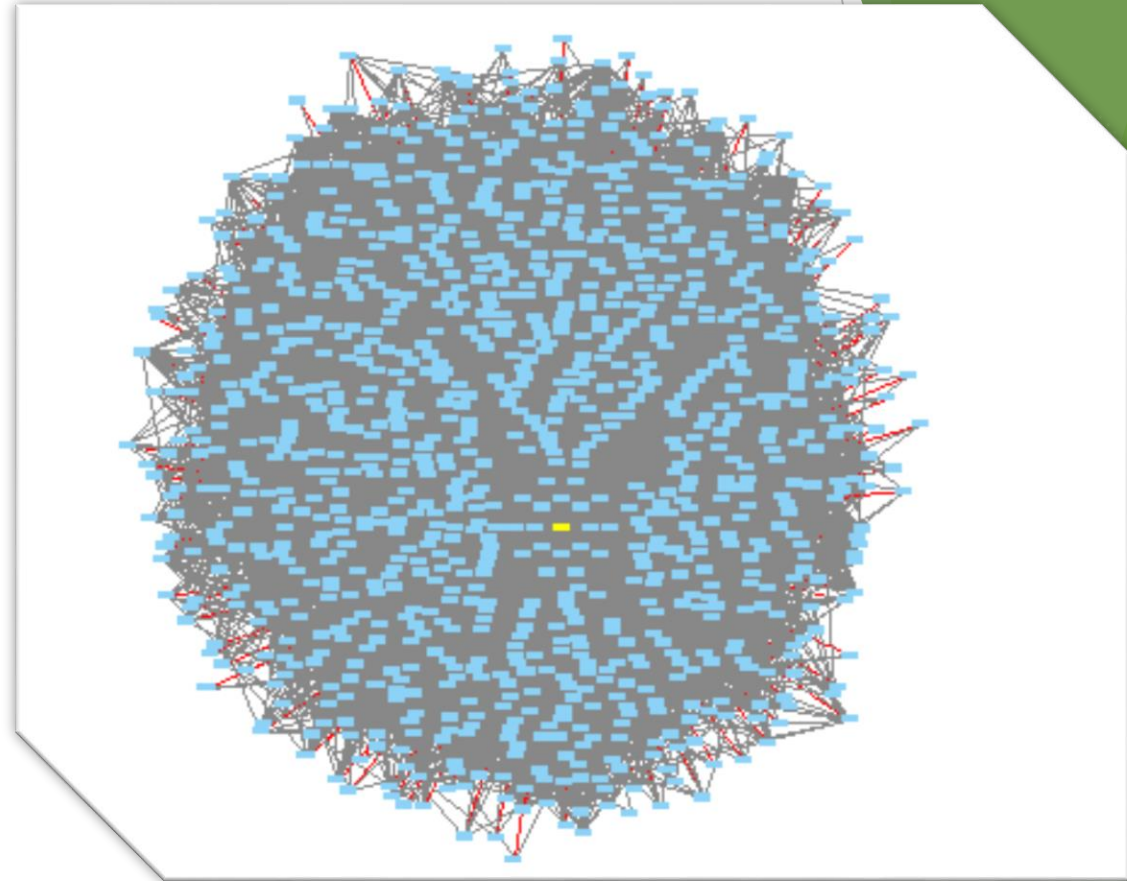
- Results:

For the eigenvector centrality (the higher the number the more important a node is)

	A	B	C	D
1		degree.g.		
2	TP53	14338	MDM2_691	0
3	MYC	13044	MAD2L1BP	0
4	TP53	10707	CCL18	0
5	LARP7	9967	GPR156	0
6	HEXIM1	9874	ZMIZ1	0
7	BRCA1	9527	E2	0
8	COP55	8761	FCAMR	0
9	CDK2	8147	GPX2	0
10	CUL7	8137	USP29	0
11	CHD3	7572	TP53INP1	0
12	CYLD	7511	USP49	0
13	RPS8	7424	APTX	0
14	RPS3A	7377	MAPK6	0

- Conclusion:

these results shows us that the node(protein) TP53(the yellow) is one of the most important proteins in our body it having a high centrality score means it plays a very big role in our protein network which in turn helps us fight off a multitude of viruses and diseases



And with that we reach the end of our project most of the files will be linked below so you can download them and see the results yourself [HERE!](#)

- Conclusion:

And if we were to look up that protein gene we will find the following

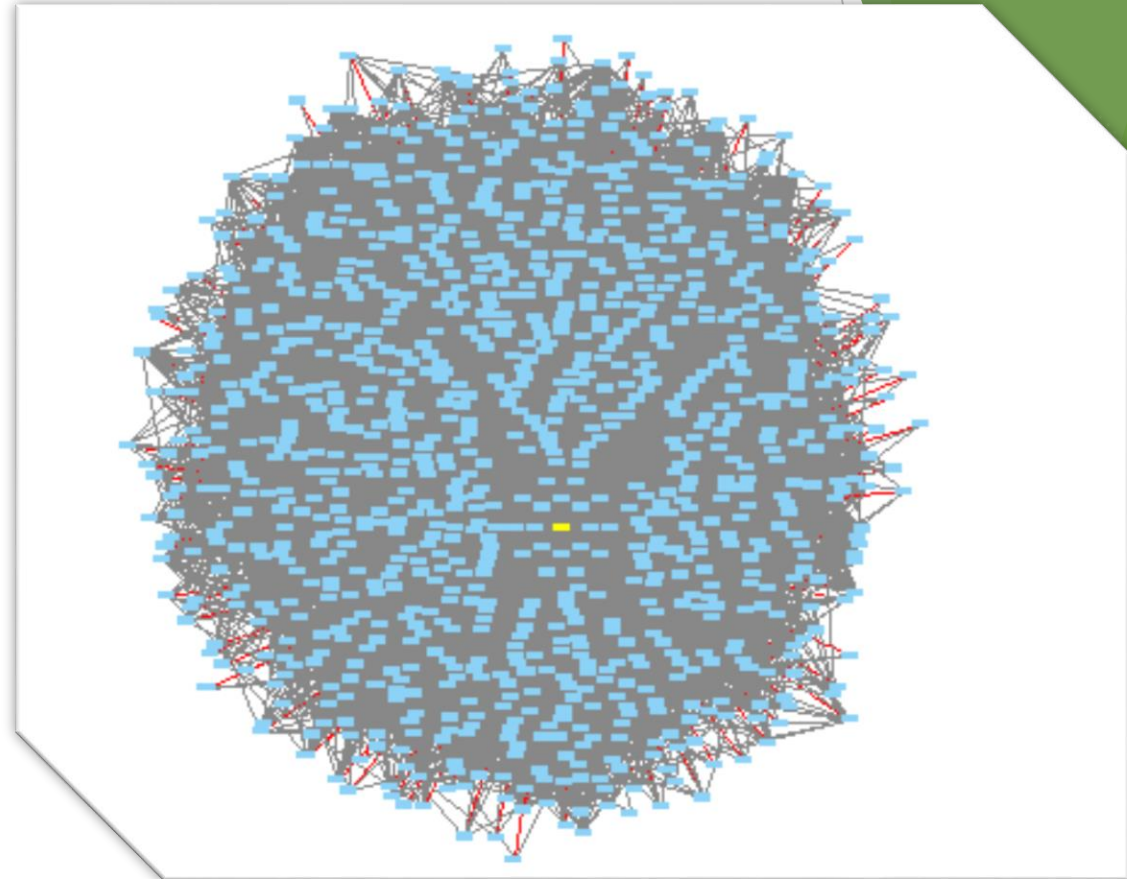
The TP53 gene provides instructions for making a protein called tumor protein p53 (or p53). This protein acts as a tumor suppressor, which means that it regulates cell division by keeping cells from growing and dividing (proliferating) too fast or in an uncontrolled way. Feb 1, 2020



MedlinePlus (.gov)

<https://medlineplus.gov> › Genetics › Genes

TP53 gene: MedlinePlus Genetics



Confirming the legitimacy of our project results

Team members: Team number 26

The students who worked on this project are:

Ahmed Mirghani Ahmed Elmakki 22011502:
Data cleaning and Code writing and Report writing
Second year

Ahmed Hussein Hassan dewdar
20225926030: Code assistant
Second year

Hamza Hussain Yousef Omran
22011501: Data set selection and management
Second year

Abdul Rahman alrimali
22011460: Data cleaning assistant
Second year

خالد ربيع عبد الجواد
22010332: dataset assistant
Second year

Samy Adel Ali
22010107:
Team management and role distribution
Second year

Ahmed Taha Mohamed
22010315: Code assistant
Second year

Kareem Mohammed Samy Aboshady
22010378: project preparation
Second year

Mohamed Ramadan Ibrahim
20221372306: project preparation
Third year